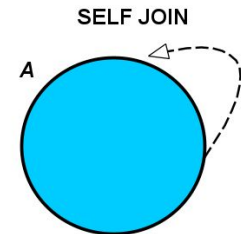
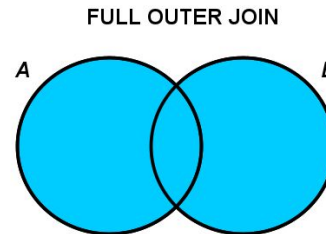
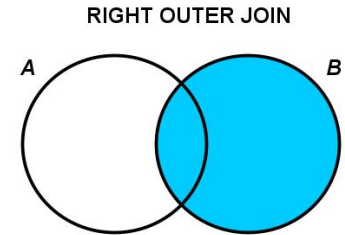
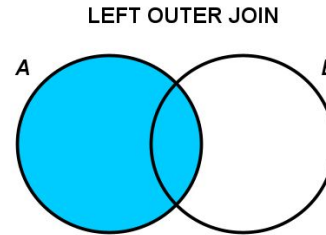
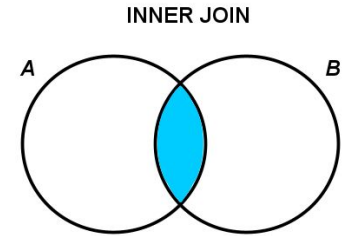
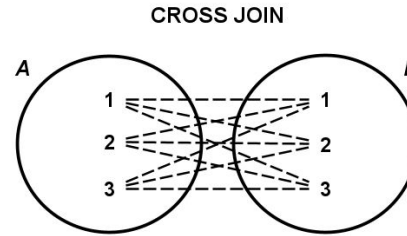


Joins

SQL joins are used to combine rows from two or more tables, based on a related column between them. Here are the main types of SQL joins:

- Inner Join
- Left Join
- Right Join
- Full Join
- Cross Join



Inner Join

Returns records that have matching values in both tables.

Syntax:

```
SELECT Customers.customer_id,  
       Customers.first_name,  
       Orders.amount  
FROM Customers  
INNER JOIN Orders  
ON Orders.customer = Customers.customer_id;
```

SQL INNER JOIN

Table: Customers

customer_id	first_name
1	John
2	Robert
<u>3</u>	David
4	John
<u>5</u>	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	<u>3</u>
3	300	6
4	800	<u>5</u>
5	150	8



customer_id	first_name	amount
3	David	500
5	Betty	800

Left Join

Returns all records from the left table (table1), and the matched records from the right table (table2). If no match, the result is NULL on the right side.

SQL LEFT JOIN

Syntax:

```
SELECT Customers.customer_id,  
       Customers.first_name,  
       Orders.amount  
FROM Customers  
LEFT JOIN Orders  
ON Orders.customer = Customers.customer_id;
```

Table: Customers

customer_id	first_name
1	John
2	Robert
3	David
4	John
5	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	3
3	300	6
4	800	5
5	150	8



customer_id	first_name	amount
1	John	
2	Robert	
3	David	500
4	John	
5	Betty	800

Right Join

Returns all records from the right table (table2), and the matched records from the left table (table1). If no match, the result is NULL on the left side.

SQL RIGHT JOIN

Syntax:

```
SELECT Customers.customer_id,  
       Customers.first_name,  
       Orders.amount  
FROM Customers  
RIGHT JOIN Orders  
ON Orders.customer = Customers.customer_id;
```

Table: Customers

customer_id	first_name
1	John
2	Robert
<u>3</u>	David
4	John
<u>5</u>	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	<u>3</u>
3	300	6
4	800	<u>5</u>
5	150	8



customer_id	first_name	amount
3	David	500
5	Betty	800
		200
		300
		150

Full Join

Returns all records when there is a match in either left (table1) or right (table2) table records.

Syntax:

```
SELECT Customers.customer_id,  
       Customers.first_name,  
       Orders.amount  
FROM Customers  
FULL OUTER JOIN Orders  
ON Orders.customer = Customers.customer_id;
```

SQL FULL OUTER JOIN

Table: Customers

customer_id	first_name
1	John
2	Robert
3	David
4	John
5	Betty

Table: Orders

order_id	amount	customer
1	200	10
2	500	3
3	300	6
4	800	5
5	150	8



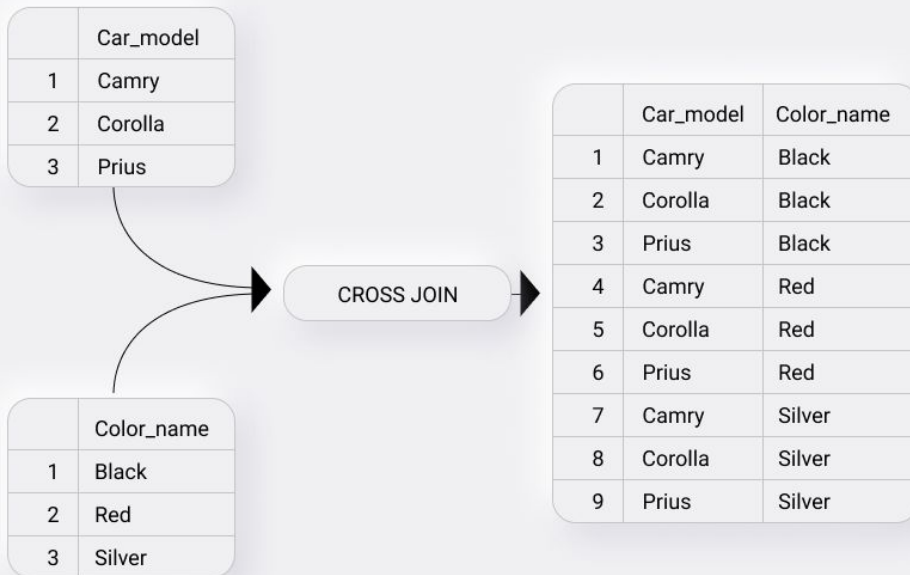
customer_id	first_name	amount
3	David	200
		500
		300
5	Betty	800
		150
2	Robert	
4	John	
1	John	

Cross Join

Returns the Cartesian product of the sets of records from the two or more joined tables when no WHERE clause is used with CROSS JOIN.

Syntax:

```
SELECT Model.car_model,  
       Color.color_name  
FROM Model  
Cross JOIN Color;
```



Self Join

A regular join, but the table is joined with itself.

Let's take a look at an example. Consider the table **Employees** :

Id	FullName	Salary	ManagerId
1	John Smith	10000	3
2	Jane Anderson	12000	3
3	Tom Lanon	15000	4
4	Anne Connor	20000	
5	Jeremy York	9000	1

Now, to show the name of the manager for each employee in the same row, we can run the following query:

```
SELECT
    employee.Id,
    employee.FullName,
    employee.ManagerId,
    manager.FullName as ManagerName
FROM Employees employee
JOIN Employees manager
ON employee.ManagerId = manager.Id
```

Id	FullName	ManagerId	ManagerName
1	John Smith	3	Tom Lanon
2	Jane Anderson	3	Tom Lanon
3	Tom Lanon	4	Anne Connor
5	Jeremy York	1	John Smith

Group By WITH ROLLUP

The GROUP BY clause in MySQL is used to group rows that have the same values in specified columns into aggregated data. The WITH ROLLUP option allows you to include extra rows that represent subtotals and grand totals.

Input Table

payment_amount	payment_date	store_id
1200.99	2018-01-18	1
189.23	2018-02-15	1
3002.43	2018-02-25	2
33.43	2018-03-03	3
7382.10	2019-01-11	2
382.92	2019-02-18	1
100.99	2019-03-07	1
322.34	2019-03-29	2
2929.14	2020-01-03	2
211.65	2020-02-02	1
499.02	2020-02-19	3
994.11	2020-03-14	1
500.73	2021-01-06	3
394.93	2021-01-22	2
3332.23	2021-02-23	3
9499.49	2021-03-10	3

- Rows with a Payment Year value but a NULL Store represent subtotals for each Payment Year.
- The row with NULL in both the Payment Year and Store columns represents the grand total payment_amount sum.

Output Table

SUM(payment_amount)	Payment Year	Store
30975.73	NULL	NULL
4426.08	2018	NULL
1390.22	2018	1
3002.43	2018	2
33.43	2018	3
8188.35	2019	NULL
483.91	2019	1
7704.44	2019	2
4633.92	2020	NULL
1205.76	2020	1
2929.14	2020	2
499.02	2020	3
13727.38	2021	NULL
394.93	2021	2
13332.45	2021	3

Group By WITH ROLLUP - Query

```
SELECT
    SUM(payment_amount),
    YEAR(payment_date) AS 'Payment Year',
    store_id AS 'Store'
FROM payment
GROUP BY YEAR(payment_date), store_id WITH ROLLUP
ORDER BY YEAR(payment_date), store_id;
```